

Report

Kristoffer Eide

Michael Birkeland

April 2026

Abstract

The purpose of our project is to identify key factors influencing predictive performance on loan defaulting and to evaluate the predictive performance of different classification models. This could provide potentially valuable insight, and a good screening system for banks when assessing loan applications. The predictors include both demographic and financial variables, such as age, income, credit score, loan type, and interest rate.

The analysis is based on a synthetic dataset containing one binary response variable indicating loan status (default or fully paid) and fifteen predictor variables describing borrower characteristics and loan attributes, where two of the predictors were added by us. We explored the data using histograms, boxplots, and barplots to understand variable distributions and relationships. Additionally, we studied class imbalance through prevalence and evaluated the relationship between the predictors using correlation analysis. Multiple classification models were implemented, including logistical regression, with and without cubic splines, k-nearest neighbors (KNN), decision trees, random forest, and boosting methods. Model performance was assessed using metrics such as misclassification rate, sensitivity, specificity, and area under PR-curves (PR AUC). Model selection and tuning were performed by implementing K-fold cross-validation.

The result shows that the tree-based models, outperform the simpler models in terms of predictive accuracy and AUC. Out of these, the random forest performed the best for our task. Key factors of loan default include loan interest rate, previous loan defaults, loan-to-income ratio, while variables such as credit score, age and education had a weaker influence. The models indicate that financial variables related to the loan and the borrower are more important predictors of default status than demographic characteristics.

Introduction

This project uses the following dataset: <https://www.kaggle.com/datasets/udaymalviya/bank-loan-data/>, which is synthetic. Each model developed will attempt to predict a borrower's loan status (i.e., whether a loan is fully paid or default) based on financial and demographic predictors, and this is thus a binary classification problem. The goal of the project is to:

- Compare the performance of multiple classification models in terms of predictive accuracy, interpretability and computational cost.
- Find the most important variables for `loan_status` and compare the demographic and financial predictors.

By answering these questions, it would hopefully give applicable models for screening loan applicants, and valuable insight in how traits affect `loan_status`, which could be useful for banks.

Data

Data wrangling

The dataset was without any NA-s, most importantly there were no NA-s for the loan_status predictor. It did however have some potential logical outliers. We identified and removed observations that satisfied at least one of these conditions: `person_age < 18` or `> 100`, `person_income < 0`, `loan_amnt ≤ 0`, `loan_int_rate ≤ 0`, `loan_percent_income < 0`, `person_emp_exp < 0` or `> person_age`, and `cb_person_cred_hist_length < 0` or `> person_age`.

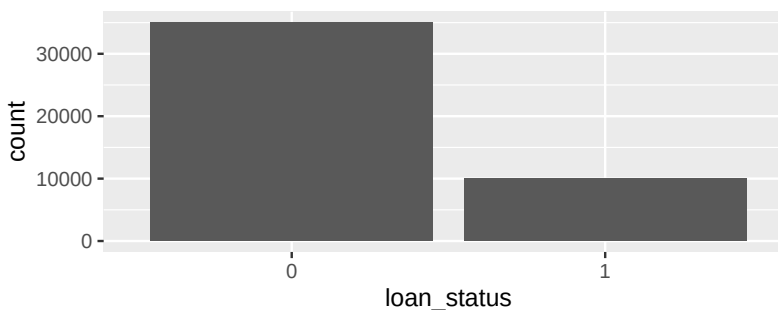
We transformed the datatype of each column that were supposed to be a categorical to the factor datatype, as this was processed as character and integer types in R. In addition we added the predictors `emp_exp_to_age` and `cred_hist_to_age`, which is employee experience to age ratio and credit history to age ratio respectively. These predictors might also be relevant for the models.

Lastly we created the train and test split. Since the default to non-default ratio is skewed (prevalence is low as seen in data analysis), we chose to stratify the splits.

Data analysis

```
## person_age person_gender person_education person_income
## Min. :20.00 female:20157 associate :12026 Min. : 8000
## 1st Qu.:24.00 male :24836 bachelor :13396 1st Qu.: 47195
## Median :26.00 doctorate : 621 Median : 67046
## Mean :27.75 high school:11970 Mean : 79908
## 3rd Qu.:30.00 master : 6980 3rd Qu.: 95778
## Max. :94.00 Max. :2448661
## person_emp_exp person_home_ownership loan_amnt
## Min. : 0.000 MORTGAGE:18485 Min. : 500
## 1st Qu.: 1.000 OTHER : 117 1st Qu.: 5000
## Median : 4.000 OWN : 2951 Median : 8000
## Mean : 5.395 RENT :23440 Mean : 9583
## 3rd Qu.: 8.000 3rd Qu.:12237
## Max. :76.000 Max. :35000
## loan_intent loan_int_rate loan_percent_income
## DEBTCONSOLIDATION:7145 Min. : 5.42 Min. :0.0000
## EDUCATION :9151 1st Qu.: 8.59 1st Qu.:0.0700
## HOMEIMPROVEMENT :4783 Median :11.01 Median :0.1200
## MEDICAL :8548 Mean :11.01 Mean :0.1397
## PERSONAL :7551 3rd Qu.:12.99 3rd Qu.:0.1900
## VENTURE :7815 Max. :20.00 Max. :0.6600
## cb_person_cred_hist_length credit_score previous_loan_defaults_on_file
## Min. : 2.000 Min. :390.0 no :22137
## 1st Qu.: 3.000 1st Qu.:601.0 yes:22856
## Median : 4.000 Median :640.0
## Mean : 5.867 Mean :632.6
## 3rd Qu.: 8.000 3rd Qu.:670.0
## Max. :30.000 Max. :784.0
## loan_status emp_exp_to_age cred_hist_to_age
## 0:34993 Min. :0.00000 Min. :0.07692
## 1:10000 1st Qu.:0.04167 1st Qu.:0.13043
## Median :0.14815 Median :0.17391
## Mean :0.16727 Mean :0.19798
## 3rd Qu.:0.26471 3rd Qu.:0.25926
## Max. :0.80851 Max. :0.58824
```

We find the prevalence of successfully paid loans (1) to defaults (0):



Prevalance: 0.2222568

The prevalence is quite low, which makes the dataset skewed. Then misclassification rate will be a less optimal measurement, as a poor baseline model always predicting the majority class will have a misclassification rate of 0.22, which is already low.

The ROCR plots the true positive ratio (TPR) against the false positive ratio (FPR), where

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

When the number of negatives is high, thus giving low prevalence, TN also tends to be high, and FPR can then be relatively low despite of FP being high. A constant low FPR, means the TPR can increase by a lot without FPR increasing too much when adjusting the decision thresholds. This gives a high ROCR curve area, and thus high AUC. The AUC baseline would be 0.5, independent of the prevalence, although low prevalence affects the ROCR curve optimistically. The ROCR can, in other words, be too optimistic despite the model producing many false positives.

We define prevalence and precision as

$$\text{prevalence} = \frac{\text{TP} + \text{FN}}{\text{TP} + \text{FN} + \text{FP} + \text{TN}}$$

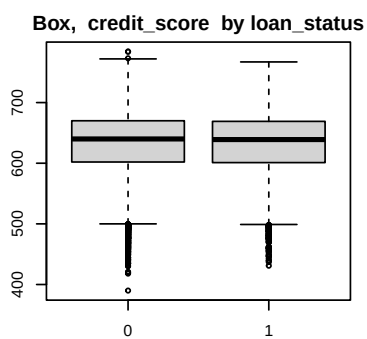
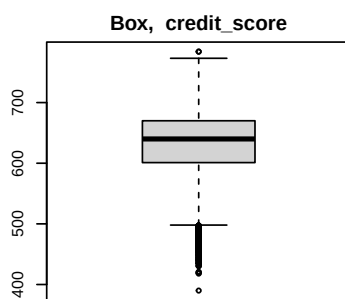
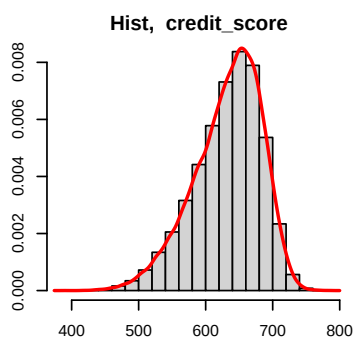
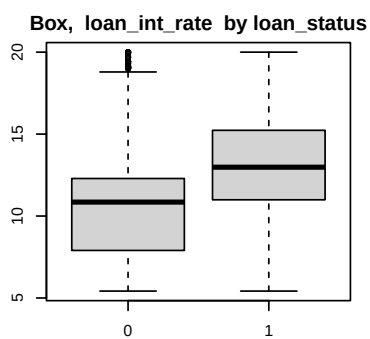
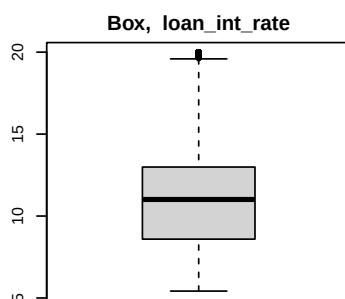
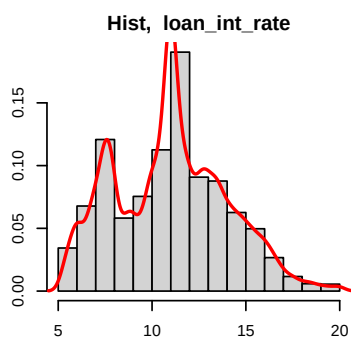
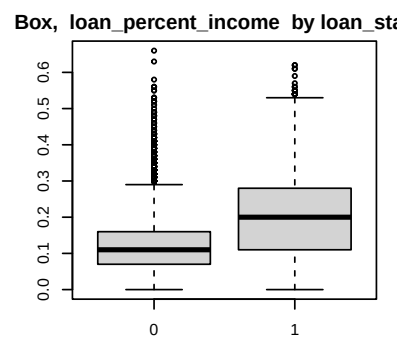
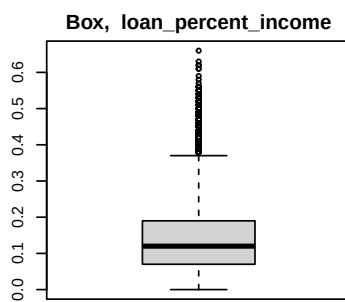
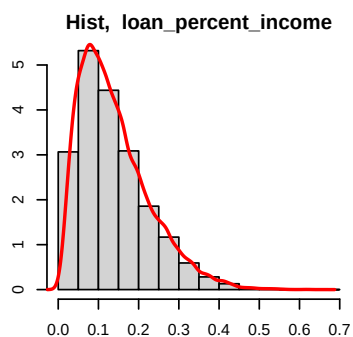
$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

The PR curve plots precision to recall (TPR). When positives is rare (TN is high, which also gives low prevalence), even a moderate number of FP would severely affect the precision. Thus giving a lower AUC score, which makes the PR sensitive to low prevalence. The baseline AUC would in this case be equal to the prevalence.

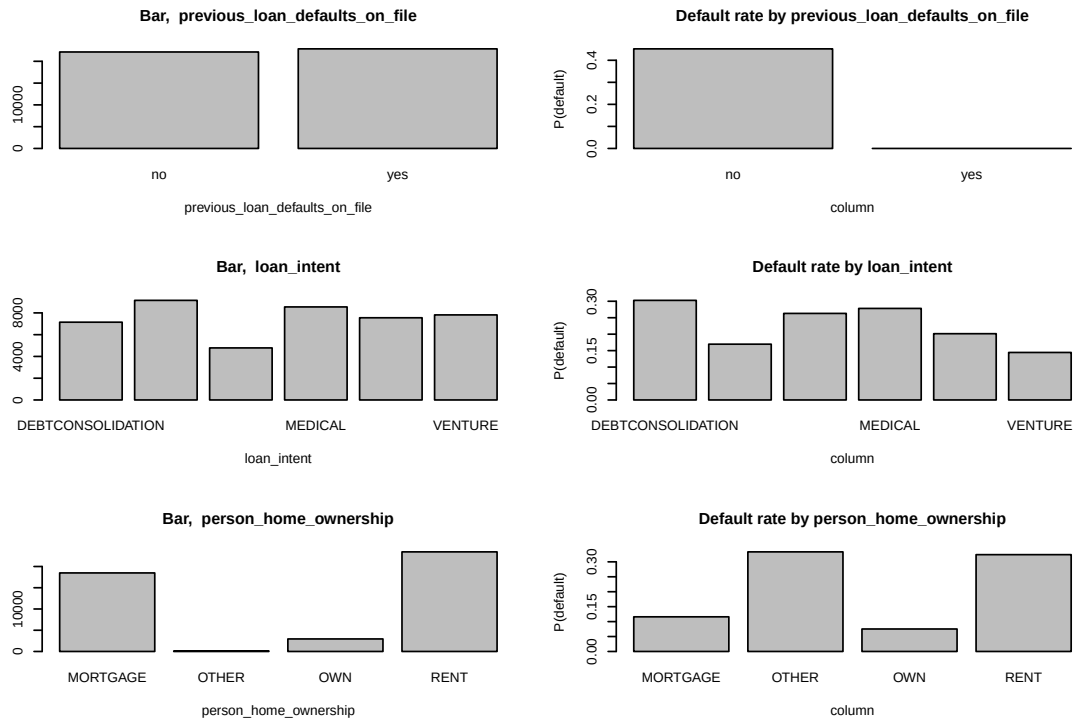
The strength of the ROCR is that it gives an interpretable result of how well the model separates the two classes, and it's baseline is not dependent on the prevalence, making it comparable across datasets. If the prevalence would be relatively even, we would therefore usually prefer the ROCR measurements. We would actually prefer the same if the prevalence is very high, as the precision then becomes very high in spite of the model producing many false positives.

Next we look at the logically most important numeric and categorical predictors. There might prove to be more important predictors later, but these are the ones we intuitively think should affect the data the most.

Numeric predictors



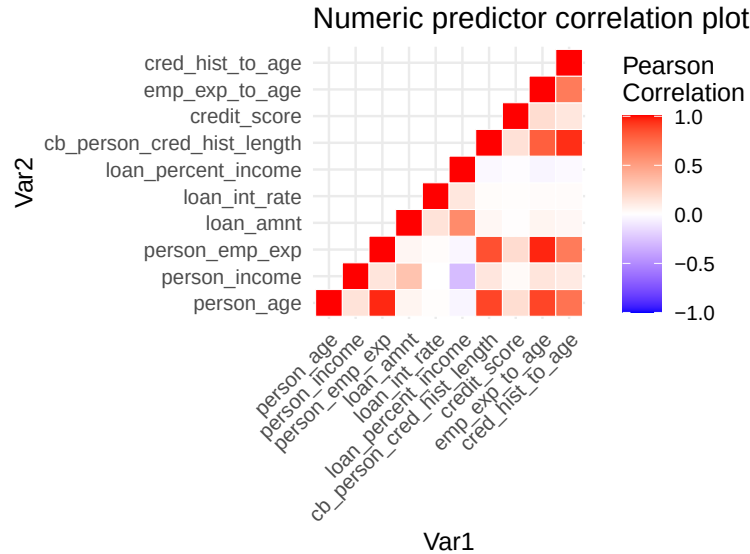
Categorical predictors



We see that `loan_percent_income` is right-skewed from the histogram and boxplot. These large outliers might have disproportionate influence on the predicted values or coefficients in KNN or log-regression, but is less dangerous for the tree based models. For our classic models, we would therefore either scale the numeric predictors for KNN or remove outliers or log transform numeric predictors for log-regression. The distribution for `loan_int_rate` seems to be multimodal, suggesting there may be hidden categories affecting the distribution. Therefore, splines could be a good idea for the classic model, while the tree models are quite insensitive to this since they are non-parametric. The `credit_score` seems pretty unimodal, but we see that unlike `loan_percent_income` and `loan_int_rate`, the boxplots do not vary much by group, indicating that this predictor might have little effect on the `loan_status`.

The `previous_loan_defaults_on_file` looks suspicious since no one who has previously defaulted defaults again. This dataset is however synthetic, which might explain the skewness. It could also be explained by there being stronger terms and conditions for people previously defaulting to get another loan, which makes them less likely to default twice. The `loan_intent` seems to have an effect on `default_rate`, as well as `person_home_ownership`. There are little observations on the categorical `other` and `own`, which can make estimated effect noisy. A possible solution would be to merge `other` and `own` into a rare group. Category `own` is however quite large, so we would prefer not to merge this into rare. Since there is then really only one rare category, the best option would perhaps be to remove all observations with `person_home_ownership = other`. We choose to keep it, to avoid losing data, since the tree models are not as affected by this, while we consider removing it for the classical models (KNN, logreg).

Since the dataset contains categorical predictors, and many of the numeric predictors do not seem to follow a Gaussian distribution, LDA and QDA seems to be less natural while KNN, log-regression and the tree models still are plausible.



The correlation plot indicates that some of the predictors are highly correlated, which is undesirable for interpretability. The model might give arbitrary weight to either predictor across different fits, resulting in large standard errors and weak interpretability. The predictors can still be very good for prediction however, as they might be engineered predictors (e.g. loan_percent_income) which gives the models clearer patterns to evaluate on. The prediction is often less hurt by correlation, as it does not care about a clean split between the weight of the correlated predictors, only the total contribution to the predicted value. To reduce the amount of correlation between the predictors we consider subset selection or regularization methods.

Models

Classical models

```
##          (Intercept)                person_age
##          -3.224286e-01                2.167850e-02
##          person_gendermale          person_educationbachelor
##          7.025253e-02                -2.936876e-02
##          person_educationdoctorate    person_educationhigh school
##          7.083772e-02                -2.361946e-02
##          person_educationmaster        person_income
##          4.334739e-03                2.005118e-06
##          person_emp_exp              person_home_ownershipOTHER
##          -9.195429e-02                2.905043e-01
##          person_home_ownershipOWN      person_home_ownershipRENT
##          -1.432799e+00                7.634317e-01
##          loan_amnt                  loan_intentEDUCATION
##          -1.092478e-04                -9.313840e-01
##          loan_intentHOMEIMPROVEMENT    loan_intentMEDICAL
##          -1.918200e-02                -2.837454e-01
##          loan_intentPERSONAL           loan_intentVENTURE
##          -7.510296e-01                -1.237358e+00
##          loan_int_rate               loan_percent_income
##          3.364189e-01                1.641736e+01
##          cb_person_cred_hist_length    credit_score
##          1.322619e-01                -8.758213e-03
```

```
## previous_loan_defaults_on_fileyes      emp_exp_to_age
##                               -2.040148e+01      1.691234e+00
##                               cred_hist_to_age
##                               -4.498246e+00
```

```
## Misclassification rate:  0.1046783
## Sensitivity:  0.753
## Specificity :  0.9359909
## AUC:  0.952224
```

```
## Misclassification rate:  0.09956662
## Sensitivity:  0.7495
## Specificity :  0.9435634
## AUC:  0.9553259
```

Decision trees

Pruned single tree

A single decision tree divides the predictor axis into several areas, on which we predict by majority vote for each observation in the area we predict on. We use a recursive binary splitting algorithm, which is computationally efficient, and can be formulated as follows

Algorithm 1: Recursive binary split

Input : stopping_rule, training_data, splitting_criterion

Output: fitted decision tree

```
1 while stopping rule not met do
2   for each possible binary split on predictors in each node do
3     | evaluate candidate split using splitting criterion
4   end
5   choose split with smallest impurity/loss
6   split the optimal node
7 end
```

The splitting criterion is RSS for regression, and node impurity for classification, either Gini index $G = \sum_{k=1}^K p_k(1 - p_k)$ or cross-entropy $D = -\sum_{k=1}^K p_k \log p_k$. Node impurity is generally better for classification, than misclassification rate. Consider as an example two classes with 400 nodes each written as (400, 400). If the model were to choose between the splits (100, 300) and (300, 100), or (200, 400) and (200, 0) the misclassification rate would be 25% for both. We would however prefer the latter, as this has a pure class (low node impurity), which is to prefer. To reduce the risk of overfitting we use a stopping rule, like minimum improvement in splitting criterion, or minimal observations in node after splitting.

To reduce the risk of overfitting, we prune the trees by minimizing a cost-complexity function. K-fold cross validation is used to find the optimal α , or equivalently the optimal subtree size. The algorithm for cost complexity and K-fold cross validation is as follows:

Algorithm 2: Pruning

Input : tree, α

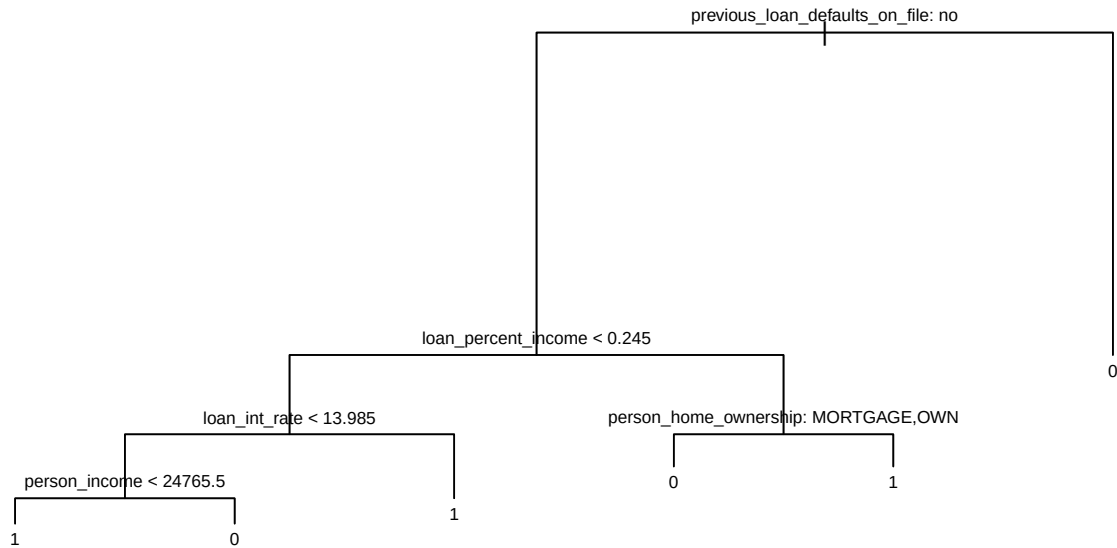
Output: pruned decision tree

```
1 minimize  $C_\alpha(T) = \alpha * |T| + Q(T)$  for different  $\alpha$ 
2 choose tree with lowest  $C_\alpha$  using cross-validation
```

Algorithm 3: K-fold cross-validation

Input : model, train_data, params, K**Output:** optimal param

```
1 divide train into K folds
2 for param_i in params do
3   for fold_j in folds do
4     train model on all folds except fold_j
5     calculate validation error  $err_{ij}$  on fold_j
6   end
7   compute mean validation error  $\bar{err}_i = \frac{1}{K} \sum_{j=1}^K err_{ij}$ 
8 end
9 return Param with lowest  $\bar{err}_i$ 
```



```
## Misclassification rate: 0.09634404
## Sensitivity: 0.696
## Specificity : 0.9629947
## AUC: 0.9402712
```

From the decision tree plot we see that the most important predictors for the tree model is:

1. previous_loan_defaults_on_file
2. loan_percent_income
3. loan_int_rate, person_home_ownership
4. person_income

Random forest

To reduce the variance of our tree models, we can use bagging. We fit multiple models on bootstrap aggregated data, then average these models. The variance of the new model is

$$Var[\bar{X}] = Var[\frac{1}{n} \sum X_i] = \frac{1}{n^2} Var[X_i] + \sum_{i \neq j} Cov(X_i, X_j)$$

Increasing n would reduce the first term, however the tree models would usually choose the same important predictors each time we fit a new model, causing the models to be correlated and thus increasing the second term. To reduce the correlation between each tree, we sample a certain fraction of the predictors during each split in the binary recursive split algorithm, only allowing the tree to choose between one of these. For classification we usually choose $m = \sqrt{p}$ and for regression $p/3$ where p is the number of predictors. The algorithm can be formulated as follows:

Algorithm 4: Random forest

Input : training_data, stopping_rule, splitting_criterion, num_predictors, num_trees

Output: fitted random forest

```

1 for  $b = 1$  to num_trees do
2   | sample nrow(training_data) bootstrap samples
3   | fit tree on the bootstrap sample using recursive binary splitting algorithm where each split considers
   |   a random subset of num_predictors predictors
4 end
5 predict by averaging each tree

```

When bootstrap aggregating the data one can show that $1/3$ of the train data is left out when training the model. On average, observation i would then not be used by $1/3 \cdot \text{num_trees}$ of the trees. Thus we can use these left out observations in each tree to calculate the error, called out of bag error, instead of using cross validation.

Algorithm 5: OOB

Input : training_data, bagging model

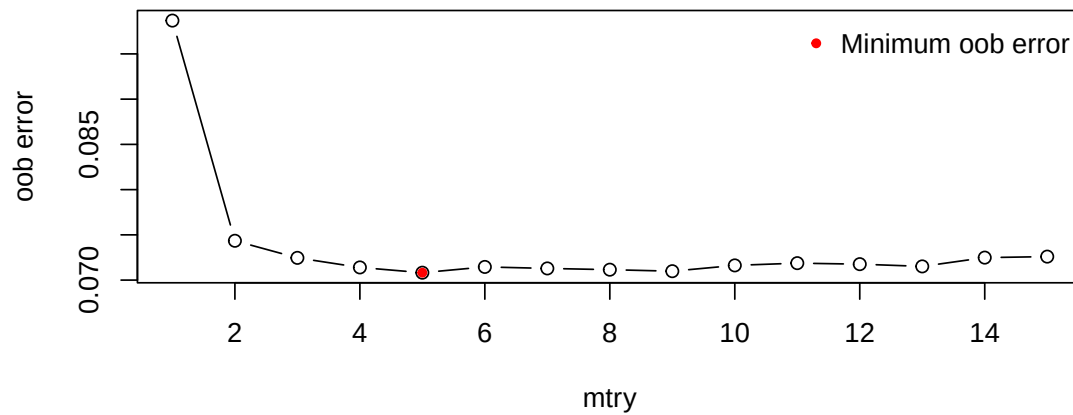
Output: oob error

```

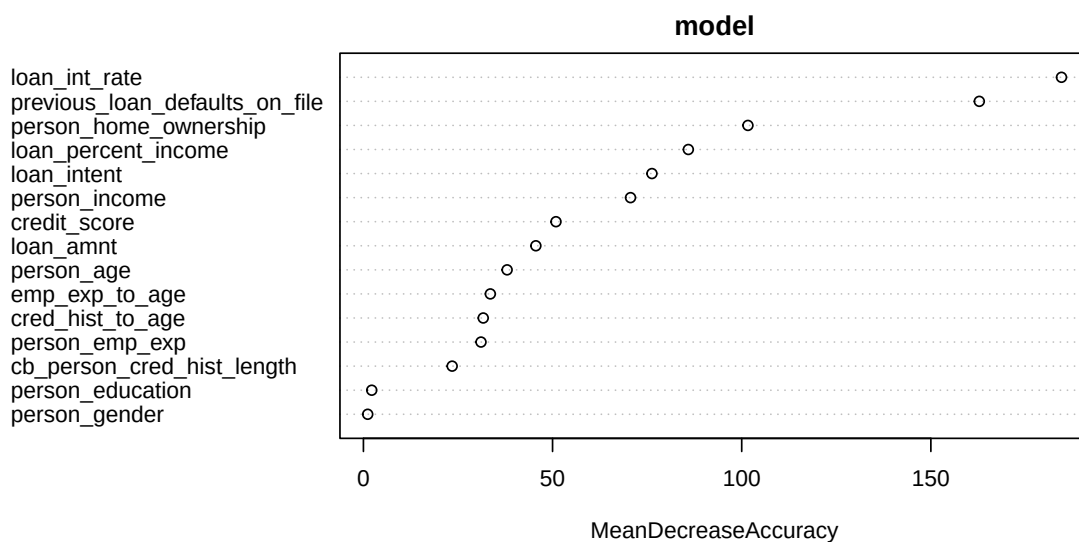
1  $n = \text{nrow}(\text{training\_data})$  for  $i=1$  to  $n$  do
2   | find each tree that excluded observation  $i$  during training
3   | predict response on observation  $i$  using these trees
4   | calculate error err $_i$ 
5 end
6 return  $(\frac{1}{n} \sum_{i=1}^n \text{err}_i)$ 

```

Although we usually would choose $\sqrt{p} = \sqrt{15} \approx 4$, we examine the best parameter selection size, mtry, using the oob error:



The optimal size for this dataset turned out to be 6. For the random forest model with $mtry = 6$, we get the following variable importance and performance:



```
## Misclassification rate: 0.07123014
## Sensitivity: 0.7675
## Specificity : 0.9748536
## AUC: 0.9742162
```

Boosting

The boosting methods are sequential, unlike bagging and random forest which builds trees independently and aggregate their predictions. In boosting, models are added one at a time, where each model attempts to improve on the previous model's error.

Ada Boost

The Ada boost algorithm can be formulated as follows

Algorithm 6: Ada boost

Input : training_data, num_trees

Output: fitted boosted model

```

1 Initialize observation weights  $w_i = 1/N, i = 1, 2, \dots, N$  Fit classifier  $G_m(x)$  on training_data using
  weights  $w_i$  for  $m = 1, 2, \dots, \text{num\_trees}$  do
2   Compute  $\text{err}_m = \frac{\sum_{i=1}^N w_i I(y_i \neq G_m(x))}{\sum_{i=1}^N w_i}$ 
3   Compute  $\alpha_m = \log(1 - \text{err}_m) / \text{err}_m$ 
4   Set  $w_i < -w_i * \exp \alpha_m \cdot I(y_i \neq G_m(x_i)), i = 1, 2, \dots, N$ 
5 end
6 return  $G(x) = \text{sign}[\sum_{m=1}^N \alpha_m G_m(x)]$ 

```

We used cross validation on the most important parameters for Ada boost, where we found the most important predictors and appropriate grids using LLM GPT-5.4. Then we got the following parameters:

```

##   iter maxdepth  nu      type      loss mean_auc
## 1    50         1 0.1 discrete exponential    NaN

```

The performance of this model was:

```

## Misclassification rate: 0.1479053
## Sensitivity: 0.396
## Specificity : 0.9824261
## AUC: 0.8857568

```

Bagged lgbm

Gradient boosting is more general than the Ada boost, allowing for optimization for any type of loss function. It can be formulated as follows:

Algorithm 7: Gradient boosting

Input : training_data, number of trees M , learning rate ν

Output: Boosted tree

```

1  $N = \text{nrow}(\text{training\_data})$ 
2 Initialize  $f_0 = \arg \min_{\gamma} \sum_{i=1}^N L(y_i, \gamma)$ 
3 for  $m = 1$  to  $M$  do
4   for  $i = 1$  to  $N$  do
5     compute  $r_{im} = -\frac{\partial L(y_i, f_{m-1}(x_i))}{\partial f(x_i)}$ 
6   end
7   fit tree on  $r_{im}$  giving terminal regions  $R_{jm}, j = 1, 2, \dots, J_m$ 
8   for  $j = 1$  to  $J_m$  do
9     compute  $\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma)$ 
10  end
11  calculate  $f_m = f_{m-1} + \nu \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$ 
12 end
13 return  $\hat{f}(x) = f_M(x)$ 

```

For classification we use the multinomial logistic loss function, which in our case is binomial:

$$p_k(x) = \frac{e^{f_k(x)}}{\sum_{l=1}^K e^{f_l(x)}}$$

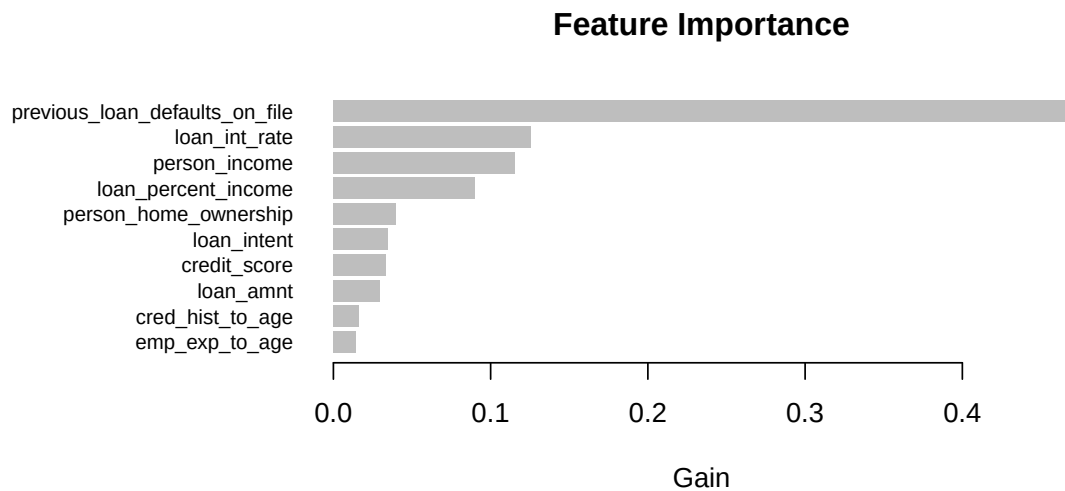
$$L(y, f(x)) = - \sum_{k=1}^K I(y = k) \log(p_k(x))$$

Lgbm additionally uses stochastic boosting, where for each tree, just like random forest, the model is only allowed to use a random subset of the predictors to reduce variance. We use cross validation to optimize the most important parameters, where we used LLM GPT-5.4 to evaluate which parameters that generally is the most important, and appropriate parameter grids for these ones. We then got the following parameters:

```
## $objective
## [1] "binary"
##
## $metric
## [1] "average_precision"
##
## $is_unbalance
## [1] TRUE
##
## $num_leaves
## [1] 31
##
## $min_data_in_leaf
## [1] 25
##
## $learning_rate
## [1] 0.05
##
## $feature_fraction
## [1] 0.8
##
## $bagging_fraction
## [1] 0.8
##
## $bagging_freq
## [1] 1
##
## $feature_pre_filter
## [1] FALSE
```

With the performance, and importance plot:

```
## Misclassification rate: 0.0791199
## Sensitivity: 0.8965
## Specificity : 0.9278468
## AUC: 0.9785075
```



To reduce the variance, we tried to bag 100 lgbm models and got the following performance:

Deviation in bootstrap:

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
## 1.878e-06 3.130e-05 1.545e-04 2.975e-02 5.128e-02 3.011e-01
```

```
## Misclassification rate: 0.07511946
## Sensitivity: 0.882
## Specificity : 0.9371339
## AUC: 0.9786177
```

The bagging did not improve the AUC significantly. A possible explanation is that the lgbm models were already quite similar, which is reflected in the low bootstrap deviance.

Summary

A summary of the performance all the models used, sorted by PR AUC, is found in the following print:

AUC:

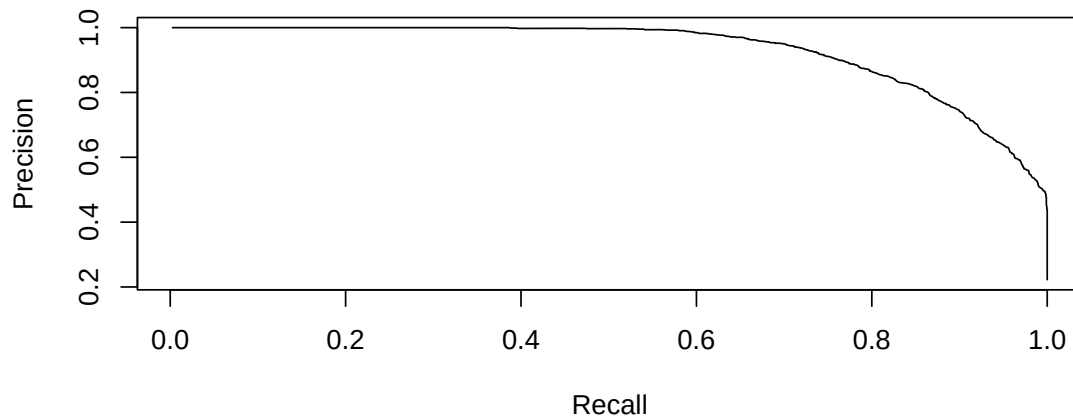
	model	misclass_rate	sensitivity	specificity	auc
## 6	bag lgbm	0.07511946	0.8820	0.9371339	0.9786177
## 5	lgbm	0.07911990	0.8965	0.9278468	0.9785075
## 3	random forest	0.07123014	0.7675	0.9748536	0.9742162
## 1	logreg	0.10467830	0.7530	0.9359909	0.9522240
## 2	prune	0.09634404	0.6960	0.9629947	0.9402712
## 4	ada boost	0.14790532	0.3960	0.9824261	0.8857568

Compared to the classical models, such as KNN and logistic regression, the tree-based methods perform substantially better in almost all aspects of the performance metrics. This is likely because these models

are able to capture complex nonlinear relationships and interactions between predictors, at the cost of interpretability, which simpler models fail to do adequately.

AUC and specificity are the most important performance metrics for our goal, combined with misclassification. This is that from a banking perspective, it would be important to catch potential defaulters (negative outcome), and we thus want the specificity to be high. This could also be achieved by simply assuming everyone defaults, thus giving a poor model, and AUC and misclassification rate will help evaluate this other aspect of the model. We see that the AUC and misclassification rate is quite high for bag lgbm, lgbm and random forest, but that the random forest performs much better with regards to the specificity. The random forest model is more interpretable than the boosting models, and computationally cheap, which is also an advantage. We would therefore prefer the random forest for this dataset, with the following performance:

```
## Confusion matrix:
##          actual
## Predicted    0    1
##           0 6822  464
##           1  177 1536
## Misclassification rate:  0.07123014
## Sensitivity:  0.768
## Specificity :  0.9747107
```



```
## AUC:  0.9742162
```

The variable importance plot for random forest, lgbm, and the decision tree shows that financial variables are the dominant predictors for loan default. Specifically, the predictor variables such as previous loan defaults, loan-to-income ratio, interest rate, income and home ownership seemed to be the most influential to the default status. On the other hand, we see there are variables with less to none influence on the loan defaulting according to the models. The demographic variables such as gender and education appear to have little predictive power. Perhaps surprisingly, variables such as credit score and loan amount also show little importance. This may be due to the synthetic nature of the data or because their effects already are captured by other correlated variables